SKRIPSI

PERBANDINGAN PROSEDUR IMPERATIF TEROTOMASI BERBASIS `KUBECTL` DAN GITOPS DENGAN ARGOCD UNTUK MENGHADAPI PENYIMPANGAN KONFIGURASI PADA KLASTER KUBERNETES

A COMPARISON OF AN AUTOMATED `KUBECTL`-BASED IMPERATIVE PROCEDURE AND ARGOCD GITOPS FOR ADDRESSING CONFIGURATION DRIFT IN KUBERNETES CLUSTERS



MUHAMMAD ARGYA VITYASY
23/522547/PA/22475

PROGRAM STUDI ILMU KOMPUTER
DEPARTEMEN ILMU KOMPUTER DAN ELEKTRONIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS GADJAH MADA
YOGYAKARTA

2026

SKRIPSI

PERBANDINGAN PROSEDUR IMPERATIF TEROTOMASI BERBASIS `KUBECTL` DAN GITOPS DENGAN ARGOCD UNTUK MENGHADAPI PENYIMPANGAN KONFIGURASI PADA KLASTER KUBERNETES

A COMPARISON OF AN AUTOMATED `KUBECTL`-BASED IMPERATIVE PROCEDURE AND ARGOCD GITOPS FOR ADDRESSING CONFIGURATION DRIFT IN KUBERNETES CLUSTERS

Diajukan untuk memenuhi salah satu syarat memperoleh derajat
S.Kom.



MUHAMMAD ARGYA VITYASY
23/522547/PA/22475

PROGRAM STUDI ILMU KOMPUTER
DEPARTEMEN ILMU KOMPUTER DAN ELEKTRONIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS GADJAH MADA
YOGYAKARTA

2026

HALAMAN PENGESAHAN

SKRIPSI

PERBANDINGAN PROSEDUR IMPERATIF TEROTOMASI BERBASIS `KUBECTL` DAN GITOPS DENGAN ARGOCD UNTUK MENGHADAPI PENYIMPANGAN KONFIGURASI PADA KLASTER KUBERNETES

Telah dipersiapkan dan disusun oleh

MUHAMMAD ARGYA VITYASY
23/522547/PA/22475

Telah dipertahankan di depan Tim Penguji
pada tanggal 17 Juni 2026

Susunan Tim Penguji

Guntur Budi Herwanto, S.Kom., M.Cs. Pembimbing	Dr. techn. Kabul Kurniawan, S.Kom., M.Cs. Ketua Penguji
	Aina Musdholifah, S.Kom., M.Kom., Ph.D. Anggota Penguji

PERNYATAAN

Dengan ini saya menyatakan bahwa dalam Skripsi ini tidak terdapat karya yang pernah diajukan untuk memperoleh gelar kesarjanaan di suatu Perguruan Tinggi, dan sepanjang pengetahuan saya juga tidak terdapat karya atau pendapat yang ditulis atau diterbitkan oleh orang lain, kecuali yang secara tertulis diacu dalam naskah ini dan disebutkan dalam daftar pustaka.

Yogyakarta, 17 Juni 2026

Muhammad Argya Vityasy

PRAKATA

Isi Prakata di sini...

DAFTAR ISI

Halaman Judul **ii**

Halaman Pengesahan **iii**

Halaman Pernyataan **iv**

Halaman Motto **v**

Halaman Persembahan **vi**

PRAKATA **vii**

DAFTAR TABEL **x**

DAFTAR GAMBAR **xi**

DAFTAR LAMBANG DAN SINGKATAN **xiii**

INTISARI **xiv**

ABSTRACT **xv**

I PENDAHULUAN **1**

1.1 Latar Belakang 1
1.2 Rumusan Masalah 2
1.3 Batasan Masalah 3
1.4 Tujuan Penelitian 3
1.5 Manfaat Penelitian 4
1.6 Sistematika Penulisan 4

II TINJAUAN PUSTAKA **5**

2.1 Penyimpangan Konfigurasi (*Configuration Drift*) 5
2.1.1 Faktor yang Menimbulkan Penyimpangan Konfigurasi 5
2.2 Terminologi Dasar 6
2.2.1 Kubernetes dan Orkestrasi Kontainer 6
2.2.2 Git dan *Version Control* 6

2.2.3 *Continuous Integration* dan *Continuous Deployment* 7
2.2.4 *Infrastructure-as-Code* (IaC) 7
2.2.5 Paradigma Deklaratif vs Imperatif 7
2.3 Manajemen Konfigurasi Infrastruktur 7
2.4 *Continuous Deployment* dan Paradigma GitOps 8
2.4.1 Performa *Deployment* dan *Anti-Pattern* 8
2.5 Dasar Teori: *Reconciliation* dan *Convergence* pada Sistem Deklaratif . 9
2.5.1 *Convergence* Menuju *Desired State* 9
2.5.2 *Audit Trail* sebagai *Social Control Mechanism* 9
2.6 *Runbook* sebagai Standarisasi Prosedur Pemulihan 9
2.7 Paradigma *Deployment* Alternatif 10
2.8 Evaluasi Empiris GitOps dan ArgoCD 10
2.9 Metode Eksperimental Terkontrol pada Riset Sistem 11
2.10 Klasifikasi Beban Uji (*Workload*) 12
2.11 Ruang Kontribusi Penelitian 12

III METODE PENELITIAN 14
3.1 Deskripsi Umum dan Status Protokol 14
3.2 Rancangan Perbandingan 14
3.3 Alur Penelitian 16
3.4 Beban Uji dan Lingkungan 17
3.4.1 Gerbang Kesiapan 17
3.5 Intervensi dan Skenario 18
3.6 Definisi Variabel dan Metrik 20
3.6.1 R1: Kesetaraan Konfigurasi dan Kesehatan 20
3.6.2 R2: Deteksi, Ketersediaan Bukti, dan Pemulihan 21
3.6.3 R3: Rubrik Bukti Keterlacakan 22
3.7 Protokol Pengulangan dan Penanganan Kegagalan 23
3.8 Pengulangan dan Analisis Data 24
3.9 Ancaman terhadap Validitas dan Keterbatasan Rancangan 25

IV HASIL DAN ANALISIS 26
4.1 Status Data dan Provenans 26
4.2 Format Pelaporan E1: Kesetaraan dan Kesehatan (R1) 26
4.3 Format Pelaporan E2: Waktu (R2) 27
4.4 Format Pelaporan E3: Keterlacakan (R3) 28

4.5 Rencana Analisis Komparatif 28
4.6 Batas Interpretasi 28

V KESIMPULAN DAN SARAN 30
5.1 Kesimpulan 30
5.2 Saran 30
5.3 Keterbatasan Penelitian 31

A LAMPIRAN A 35
1.1 Artefak Eksperimen yang Direncanakan 35
1.2 Skema Data dan Perlindungan Informasi 35
1.3 Status Lampiran 35

DAFTAR TABEL

3.1 Skenario intervensi yang direncanakan 19
3.2 Rubrik penilaian bukti R3 22

4.1 Skema hasil R1 per skenario dan lingkungan 27
4.2 Skema hasil R2 per skenario dan lingkungan 27

DAFTAR GAMBAR

3.1 Rancangan perbandingan dua prosedur otomatis dengan evaluator dan definisi metrik yang sama. 15
3.2 Bagan alir protokol, dengan pilot dipisahkan dari pengulangan formal. 16

DAFTAR LAMBANG DAN SINGKATAN

ArgoCD Alat GitOps *pull-based* yang melakukan *continuous reconciliation* terhadap *state* klaster Kubernetes

GitOps Paradigma manajemen *deployment* yang menggunakan repositori Git sebagai *single source of truth* dengan agen *reconciliation* otomatis

IaC *Infrastructure-as-Code*: praktik mendefinisikan dan mengelola infrastruktur melalui berkas konfigurasi yang diberi versi

R1 Metrik konsistensi *deployment*

R2 Metrik waktu deteksi, ketersediaan bukti penyimpangan, dan pemulihan konfigurasi

R3 Metrik *traceability* operasional (deskriptif)

INTISARI

Perbandingan Prosedur Imperatif Terotomasi Berbasis `kubectl` dan GitOps dengan ArgoCD untuk Menghadapi Penyimpangan Konfigurasi pada Klaster Kubernetes

Oleh

Muhammad Argya Vityasy
23/522547/PA/22475

Skripsi ini merancang evaluasi empiris atas prosedur imperatif terotomasi berbasis `kubectl` dan GitOps menggunakan ArgoCD dalam menghadapi penyimpangan konfigurasi pada Kubernetes. Pengujian formal direncanakan pada beban uji multi-service InvenioRDM; surrogat S1 dan S2 hanya digunakan dalam pilot validasi instrumen dan tidak digabungkan dengan data formal. HPA normal dipertahankan pada kedua lengan, sehingga skenario A menjadi kontrol HPA yang dilaporkan terpisah dari efikasi ArgoCD; skenario F diperlakukan terpisah sebagai batas visibilitas sumber daya di luar cakupan aplikasi. Metrik yang dirancang mencakup kesetaraan bidang konfigurasi dan kesehatan secara terpisah (R1), waktu deteksi, waktu ketersediaan bukti penyimpangan, dan waktu pemulihan konfigurasi penuh (R2), serta rubrik bukti keterlacakan (R3). Target pengulangan adalah lima run per skenario dan lingkungan untuk R1 serta sepuluh untuk R2; angka ini merupakan target deskriptif, bukan jaminan kecukupan daya. Protokol dan apparatus masih menunggu tinjauan serta validasi, dan data formal belum tersedia. Oleh sebab itu, skripsi ini belum melaporkan hasil komparatif atau kesimpulan empiris.

Kata Kunci: penyimpangan konfigurasi, Kubernetes, GitOps, otomasi imperatif, evaluasi empiris terkontrol

ABSTRACT

A Comparison of an Automated `kubectl`-Based Imperative Procedure and ArgoCD GitOps for Addressing Configuration Drift in Kubernetes Clusters

By

Muhammad Argya Vityasy
23/522547/PA/22475

This thesis drafts an empirical evaluation of an automated imperative procedure based on `kubectl` and GitOps using ArgoCD for configuration drift in Kubernetes. Formal testing is planned on the InvenioRDM multi-service workload; S1 and S2 surrogates are reserved for instrument-validation pilots and will not be pooled with formal data. Normal HPA operation is retained in both arms, so scenario A is an HPA control reported separately from ArgoCD efficacy, while scenario F is treated as a visibility-boundary case for resources outside the application scope. The measures comprise configuration-field equality and health reported separately (R1), time to detection, availability of verifiable deviation evidence, and full configuration restoration (R2), and an evidence-based traceability rubric (R3). Planned sample targets are five runs per scenario and environment for R1 and ten for R2; these are descriptive targets, not guarantees of statistical power. The protocol and apparatus remain subject to review and validation, and no formal dataset is available. Accordingly, no comparative findings or empirical conclusions are reported in this draft.

Keywords: configuration drift, Kubernetes, GitOps, automated imperative management, controlled empirical evaluation

BAB I

PENDAHULUAN

1.1 Latar Belakang

Kubernetes mengelola sumber daya aplikasi melalui keadaan yang dideklarasikan dan rekonsiliasi oleh komponen pengendali. Dalam operasi nyata, keadaan aktual dapat berbeda dari deklarasi yang menjadi acuan. Perbedaan ini disebut penyimpangan konfigurasi (*configuration drift*). Penyimpangan perlu dibedakan dari cacat statis: suatu manifest dapat mengandung konfigurasi yang keliru tetapi tetap sama dengan keadaan aktual, sedangkan drift merujuk pada ketidakcocokan antara acuan dan keadaan yang berjalan. Dengan demikian, pengukuran kesetaraan konfigurasi tidak dengan sendirinya menilai apakah nilai konfigurasi tersebut baik atau aman (Zhang et al., 2026; Rahman et al., 2023).

Pengelolaan deklaratif dapat menggunakan pengendali GitOps untuk membandingkan sumber daya klaster dengan deklarasi di repositori dan mencoba merekonsiliasikannya. Pendekatan imperatif menggunakan perintah untuk mengubah keadaan klaster; perilaku setelah perubahan bergantung pada otomasi dan prosedur yang dipasang. Karena itu, perbandingan empiris perlu menetapkan dengan jelas mekanisme yang dibandingkan. Penelitian ini tidak mengukur kecepatan atau keahlian manusia. Lingkungan pembanding imperatif menggunakan prosedur otomatis berbasis `kubectl` yang tetap dan terstandarisasi, sementara lingkungan lain menggunakan ArgoCD dengan kebijakan rekonsiliasi yang akan diverifikasi sebelum eksperimen.

Sejumlah penelitian telah mengevaluasi GitOps maupun membandingkannya dengan pendekatan lain. Shrestha dan Ali membahas waktu induksi, penemuan, dan pemulihan drift dalam perbandingan GitOps dengan Ansible (Shrestha and Ali, 2024). Kagganti Nataraja mendeskripsikan perbandingan terkontrol pada Minikube satu node dengan konfigurasi keamanan yang berbeda (Nataraja, 2025). Matubber juga memasukkan pembanding lama yang semi-terotomasi dalam evaluasi infrastruktur Kubernetes (Matubber, 2025). Studi-studi tersebut menunjukkan bahwa evaluasi komparatif telah dilakukan; penelitian ini tidak mengklaim sebagai perbandingan pertama atau menganggap bahwa GitOps pasti lebih unggul. Rincian lingkup dan batas interpretasi studi tersebut dibahas pada Bab II.

Penelitian ini merancang pengukuran per skenario pada beban uji multi-

service InvenioRDM. Pengukuran membedakan kesetaraan bidang konfigurasi dari kesehatan layanan, memisahkan waktu deteksi dari ketersediaan bukti yang dapat diverifikasi, dan mengukur pemulihan terhadap kesetaraan inventaris serta bidang yang telah ditetapkan. Keterlacakan dinilai berdasarkan bukti yang tersedia, bukan asumsi bahwa suatu alat selalu menghasilkan jejak lengkap. Skenario F diperlakukan secara terpisah karena menguji batas visibilitas sumber daya di luar cakupan aplikasi, bukan keberhasilan self-heal atas sumber daya terkelola.

Rancangan ini dimaksudkan untuk memperjelas perbedaan operasional antara dua prosedur yang ditetapkan pada konteks eksperimen tertentu. Rekomendasi praktis maupun kesimpulan tentang arah perbedaan hanya dapat dibuat setelah apparatus divalidasi, protokol ditinjau, dan data formal dikumpulkan. Pada draf ini, hasil eksperimen belum tersedia.

1.2 Rumusan Masalah

Penelitian ini diarahkan oleh pertanyaan berikut.

1. **RQ1:** Bagaimana kesetaraan bidang konfigurasi yang ditetapkan dan kesehatan layanan, yang dilaporkan secara terpisah, berubah setelah intervensi dan prosedur koreksi pada prosedur imperatif terotomasi berbasis `kubectl` dibandingkan dengan GitOps menggunakan ArgoCD?

2. **RQ2:** Bagaimana distribusi waktu deteksi, waktu hingga tersedia bukti penyimpangan yang dapat diverifikasi, dan waktu pemulihan konfigurasi penuh berbeda antara kedua prosedur untuk setiap skenario yang dianalisis?

3. **RQ3:** Bukti apa yang tersedia untuk merekonstruksi pelaku, objek dan bidang yang berubah, waktu serta urutan kejadian, dan tindakan pemulihan pada masing-masing prosedur?

RQ1 dan RQ2 dijawab per skenario. HPA normal tetap aktif pada kedua lengan; A menjadi kontrol HPA, bukan bukti efikasi ArgoCD memulihkan replika yang diabaikan. A dan uji batas visibilitas F dilaporkan terpisah dari estimasi B–E/G. Tidak adanya OutOfSync bukan waktu deteksi nol. RQ3 bersifat deskriptif dan tidak mengukur waktu identifikasi akar masalah. Penyesuaian tersebut tetap draf untuk tinjauan pemilik dan pembimbing; eksekusi formal dinonaktifkan.

1.3 Batasan Masalah

Ruang lingkup penelitian dibatasi sebagai berikut.

1. Perbandingan mencakup satu prosedur imperatif terotomasi berbasis `kubectl` dan satu implementasi GitOps berbasis ArgoCD. Tidak ada intervensi manusia dalam waktu deteksi atau pemulihan yang diukur; hasil tidak mewakili kecepatan operator atau semua implementasi kedua paradigma.

2. Beban uji formal direncanakan berupa workload komposit multi-service InvenioRDM pada satu klaster Kubernetes. Komposisi aktual, versi, dan kapasitas lingkungan harus diverifikasi sebelum pengujian. Generalisasi dibatasi pada konfigurasi apparatus yang dilaporkan.

3. Surrogat S1 dan S2 hanya digunakan pada pilot untuk memvalidasi instrumen dan prosedur. Data pilot tidak menggantikan beban formal, tidak dipool dengan data formal, dan tidak mendukung kesimpulan tentang InvenioRDM.

4. Intervensi yang direncanakan meliputi perubahan replika, image, ConfigMap, penghapusan sumber daya, patch bidang terkelola, sumber daya pada namespace di luar cakupan aplikasi (F), dan perubahan Secret. F dianalisis sebagai batas visibilitas sumber daya tak terkelola; bukan sebagai skenario self-heal yang diasumsikan.

5. R1 menilai kesetaraan inventaris dan bidang deklaratif yang ditetapkan dengan aturan sama pada kedua lingkungan; kesehatan dilaporkan terpisah. R2 mencakup waktu deteksi, ketersediaan bukti penyimpangan, dan pemulihan konfigurasi penuh. R3 menggunakan rubrik bukti deskriptif. Nilai Secret tidak dipublikasikan.

6. Analisis dibatasi pada hasil per skenario. Target jumlah pengulangan merupakan target deskriptif, bukan jaminan kecukupan daya. Data tersensor, kegagalan injeksi, dan kegagalan instrumentasi dilaporkan terpisah; tidak ada nilai hilang yang diganti nol.

1.4 Tujuan Penelitian

Penelitian ini bertujuan merancang dan, setelah protokol disetujui serta apparatus divalidasi, mengevaluasi perbedaan operasional antara prosedur imperatif tero-

tomasi berbasis `kubectl` dan GitOps berbasis ArgoCD pada beban uji formal multi-service InvenioRDM. Evaluasi mengoperasionalkan RQ1 melalui kesetaraan konfigurasi dan kesehatan yang terpisah, RQ2 melalui waktu deteksi, bukti, serta pemulihan, dan RQ3 melalui rubrik ketersediaan bukti keterlacakan. Tujuan ini tidak menyatakan sebelumnya prosedur mana yang lebih baik.

1.5 Manfaat Penelitian

Jika protokol berhasil divalidasi dan data dikumpulkan, penelitian ini diharapkan dapat:

1. memberikan deskripsi empiris yang dibatasi pada workload, skenario, dan konfigurasi yang benar-benar diuji;
2. mendokumentasikan definisi pengukuran serta aturan penanganan data tersensor agar dapat ditinjau dan direplikasi; dan
3. menyediakan informasi kontekstual bagi praktisi tanpa mengubah hasil satu apparatus menjadi rekomendasi universal tentang GitOps atau pengelolaan imperatif.

1.6 Sistematika Penulisan

Skripsi ini disusun sebagai berikut:

- Bab I menjelaskan latar belakang, rumusan masalah, batasan, tujuan, manfaat, dan sistematika penulisan.
- Bab II membahas konsep drift, pendekatan pengelolaan konfigurasi, studi empiris terkait, serta ruang kontribusi yang dibatasi.
- Bab III menjelaskan rancangan perbandingan, workload, skenario, definisi metrik, protokol, analisis, dan ancaman validitas.
- Bab IV menyediakan struktur pelaporan setup dan bukti untuk R1–R3; hasil empiris hanya dapat diisi setelah eksperimen formal tervalidasi.
- Bab V merangkum status kesimpulan, keterbatasan, serta rekomendasi yang dapat ditindaklanjuti setelah hasil tersedia.

BAB II

TINJAUAN PUSTAKA

2.1 Penyimpangan Konfigurasi (*Configuration Drift*)

Penyimpangan konfigurasi (*configuration drift*) adalah ketidakcocokan antara keadaan aktual sistem dan keadaan yang ditetapkan sebagai acuan deklaratif (Pohjola, 2025). Drift berbeda dari cacat statis: konfigurasi yang tidak aman dapat tetap cocok dengan acuannya, sedangkan perubahan aktual yang tidak cocok dengannya merupakan drift meskipun nilai aktual belum dinilai dampaknya. Penelitian ini mengukur kesetaraan terhadap deklarasi, bukan kebenaran semantik atau keamanan seluruh manifest.

Penelitian empiris oleh Zhang et al. (2026) melakukan studi terhadap 719 *defect* konfigurasi dari 2.260 skrip Kubernetes dan mengidentifikasi 15 kategori *defect*, di mana 533 dari 719 *defect* (74,1%) menyebabkan *crash*, operasi yang salah, atau *outage*. Studi tersebut menegaskan bahwa *misconfiguration* merupakan risiko operasional utama pada Kubernetes. Rahman et al. (2023) menambahkan bahwa *security misconfiguration* pada manifest Kubernetes *open-source* mencakup 11 kategori yang berulang, termasuk *container* yang berjalan sebagai *root*, tanpa *drop capabilities*, dan tanpa *resource limits*. Penelitian menunjukkan bahwa penyimpangan konfigurasi bukan hanya masalah estetika konfigurasi, tetapi ancaman nyata terhadap keandalan dan keamanan layanan.

Pohjola (2025) memberikan kerangka konseptual untuk memahami sumber dan dampak penyimpangan pada sistem *Infrastructure-as-Code* (IaC). Penelitian tersebut mengidentifikasi tiga faktor pendorong utama penyimpangan: (1) keahlian manusia (*human expertise*) yang tidak konsisten; (2) tekanan waktu (*time pressure*) yang memicu perubahan cepat dan tidak didokumentasikan; dan (3) ketiadaan mekanisme deteksi otomatis pada *pipeline deployment*. Meskipun penelitian tersebut tidak beroperasi pada Kubernetes secara eksklusif, prinsip-prinsip yang dikemukakan berlaku umum untuk platform orkestrasi berbasis konfigurasi deklaratif.

2.1.1 Faktor yang Menimbulkan Penyimpangan Konfigurasi

Berdasarkan literatur, faktor-faktor utama yang memperburuk penyimpangan konfigurasi pada Kubernetes meliputi:

1. Perubahan langsung pada klaster: Penggunaan `kubectl edit`, `kubectl patch`, atau `kubectl scale` secara langsung tanpa memperbarui repositori *source of truth* (Zhang et al., 2026).

2. Prosedur *deployment* yang tidak terdokumentasi: Setiap *operator* dapat memiliki urutan perintah yang berbeda, yang mengakibatkan *state* klaster tidak deterministik (Pohjola, 2025).

3. Kurangnya mekanisme *rollback* yang terintegrasi: kemampuan untuk mengembalikan perubahan bergantung pada pencatatan versi, ketersediaan nilai dasar, dan fitur yang dikonfigurasi pada alat pengelolaan (Rahman et al., 2023).

4. Ketiadaan mekanisme deteksi penyimpangan otomatis: Tanpa pemeriksaan berkala antara *state* aktual dan *desired state*, penyimpangan dapat terakumulasi selama periode lama tanpa terdeteksi (Zhang et al., 2026).

2.2 Terminologi Dasar

Bagian ini mendefinisikan terminologi kunci yang digunakan dalam penelitian, dengan tujuan membangun dasar konseptual yang konsisten sebelum pembahasan lebih lanjut.

2.2.1 Kubernetes dan Orkestrasi Kontainer

Kubernetes adalah platform *container orchestration open-source* yang mengotomasi *deployment*, *scaling*, dan manajemen aplikasi terkemas (*containerized*) (Burns et al., 2016). Kubernetes awalnya dikembangkan oleh Google berdasarkan pengalaman mengelola sistem *workload* berskala besar di Borg dan Omega, dan kini menjadi standar de facto untuk orkestrasi kontainer di industri (Verma et al., 2015). Pada Kubernetes, *state* yang diinginkan (*desired state*) dideklarasikan melalui berkas konfigurasi (*manifest*), dan *control plane* bertanggung jawab mencapai dan mempertahankan *state* tersebut.

2.2.2 Git dan *Version Control*

Git adalah sistem *version control* terdistribusi yang merekam setiap perubahan pada berkas beserta metadata (penulis, *timestamp*, pesan *commit*) (Chacon and Straub, 2014). Dalam konteks GitOps, Git berfungsi sebagai *single source of truth*

yang menyimpan deklarasi *state* infrastruktur dan aplikasi, sehingga setiap perubahan dapat dilacak, di-*review*, dan dikembalikan (*rollback*) secara andal.

2.2.3 *Continuous Integration* dan *Continuous Deployment*

Continuous Integration (CI) adalah praktik di mana perubahan kode secara otomatis diuji dan digabungkan ke *branch* utama, sementara *Continuous Deployment* (CD) adalah perluasan CI di mana perubahan yang lulus pengujian secara otomatis di-*deploy* ke lingkungan produksi tanpa intervensi manual eksplisit (Humble and Farley, 2010). GitOps merupakan paradigma spesifik di dalam CD yang menggunakan repositori Git sebagai *single source of truth*.

2.2.4 *Infrastructure-as-Code* (IaC)

Infrastructure-as-Code (IaC) adalah praktik mendefinisikan dan mengelola infrastruktur melalui berkas konfigurasi yang diberi versi (*versioned*), bukan melalui proses manual (Morris, 2020). IaC memungkinkan *version control*, *code review*, dan reprodusibilitas lingkungan. Dalam konteks Kubernetes, IaC diimplementasikan melalui *manifest* deklaratif (YAML) yang mendefinisikan *desired state* klaster.

2.2.5 Paradigma Deklaratif vs Imperatif

Dalam paradigma imperatif, tindakan pengelolaan dinyatakan sebagai perintah atau urutan tindakan yang mengubah keadaan sistem (*how to achieve the state*). Dalam paradigma deklaratif, keadaan yang diinginkan dinyatakan dan pengendali bertugas mendekatinya (*what state to achieve*) (Scott, 2015). Keduanya dapat diotomasi; perbedaan untuk penelitian ini adalah prosedur koreksi imperatif tetap yang dijalankan melalui `kubectl` dibandingkan dengan rekonsiliasi ArgoCD.

2.3 Manajemen Konfigurasi Infrastruktur

Infrastructure state management mengatur bagaimana keadaan infrastruktur didefinisikan, diterapkan, dan dipertahankan. Pendekatan deklaratif menyimpan keadaan yang diinginkan dalam konfigurasi, sedangkan pendekatan imperatif menyatakan tindakan untuk mengubah keadaan (Morris, 2020; Limoncelli et al., 2018). Otomasi, pemeriksaan berulang, dan pencatatan dapat tersedia pada kedua pendekatan; karakteristik tersebut perlu ditetapkan pada tingkat implementasi yang dibandingkan.

2.4 *Continuous Deployment* dan Paradigma GitOps

Continuous Deployment (CD) adalah praktik dalam ranah DevOps di mana setiap perubahan kode yang lulus pengujian secara otomatis di-*deploy* ke lingkungan produksi tanpa intervensi manual eksplisit (Limoncelli et al., 2018). CD merupakan perluasan dari *Continuous Integration* (CI) dan bertujuan meminimalkan waktu antara penulisan kode dan ketersediaannya bagi pengguna akhir. Dalam ekosistem Kubernetes, CD memerlukan mekanisme yang dapat secara andal menyelaraskan perubahan konfigurasi dengan *state* klaster.

GitOps adalah paradigma spesifik di dalam CD yang menggunakan repositori Git sebagai *single source of truth* untuk mendefinisikan *state* infrastruktur dan aplikasi. CNCF GitOps Working Group mendefinisikan empat prinsip OpenGitOps: (1) sistem dideklarasikan secara deklaratif; (2) *state* sistem yang diinginkan tersimpan dan diberi versi (*versioned*) dalam Git; (3) perubahan *state* ditarik (*pulled*) secara otomatis oleh agen *software*; dan (4) agen *software* secara kontinu mendeteksi dan mengoreksi deviasi dari *state* yang diinginkan (CNCF GitOps Working Group, 2021).

GitOps mendukung pencatatan versi deklarasi melalui Git, peninjauan perubahan, dan penggunaan revisi sebelumnya sebagai acuan pemulihan (Yuen et al., 2021). Riwayat Git tidak otomatis merekam perubahan langsung pada klaster; tindakan tersebut memerlukan bukti dari pengendali atau audit API. Pada model *pull-based*, agen di dalam klaster dapat menarik deklarasi tanpa pipeline eksternal menerapkan setiap perubahan ke API klaster, tetapi kebutuhan kredensial dan akses operasional tetap bergantung pada rancangan serta konfigurasi sistem (Utami and Fatta, 2021).

2.4.1 Performa *Deployment* dan *Anti-Pattern*

Forsgren dkk. (Forsgren et al., 2018) membahas *deployment frequency*, *lead time for changes*, *time to restore service*, dan *change failure rate* sebagai metrik performa pengembangan dan operasi. Metrik organisasi tersebut tidak identik dengan pengukuran eksperimen ini dan tidak dengan sendirinya membuktikan bahwa satu prosedur selalu lebih unggul. Penelitian ini menggunakan konstruk yang lebih terbatas: kesetaraan bidang konfigurasi, waktu deteksi dan pemulihan konfigurasi, serta bukti keterlacakan.

ArgoCD menyediakan pengendali aplikasi yang membandingkan sumber daya teramati dengan deklarasi repositori dan, jika kebijakan serta cakupannya mengizinkan, dapat melakukan sinkronisasi atau koreksi otomatis (Argo Project, 2024).

Perilaku aktual bergantung pada konfigurasi aplikasi, izin, ketersediaan pengendali, jenis sumber daya, dan cakupan inventaris. Oleh sebab itu, rekonsiliasi tidak menjamin setiap perubahan akan selalu ditemukan atau dipulihkan.

2.5 Dasar Teori: *Reconciliation* dan *Convergence* pada Sistem Deklaratif

Karakteristik GitOps yang berbeda dari pendekatan imperatif dapat dijelaskan melalui dua prinsip teoretis yang saling berkaitan.

2.5.1 *Convergence* Menuju *Desired State*

Pengendali deklaratif secara berkala membandingkan keadaan aktual dengan keadaan yang diinginkan dan dapat melakukan tindakan korektif sesuai kebijakan yang berlaku (Argo Project, 2024). Keberhasilan konvergensi bukan sifat yang dapat diasumsikan untuk semua sumber daya atau gangguan: hasil dipengaruhi oleh cakupan aplikasi, izin, ketersediaan komponen, konfigurasi sinkronisasi, dan sifat sumber daya.

Perintah imperatif tidak dengan sendirinya menyediakan rekonsiliasi berkelanjutan. Namun, prosedur imperatif dapat diotomasi dengan pemeriksaan berkala dan tindakan koreksi yang telah ditentukan. Karena penelitian ini membandingkan prosedur `kubectl` terotomasi dengan ArgoCD, bukan respons manusia, perbedaan waktu tidak ditafsirkan sebagai perbedaan kecepatan operator.

2.5.2 *Audit Trail* sebagai *Social Control Mechanism*

Riwayat Git dapat menyediakan metadata dan perubahan deklarasi, sedangkan log pengendali dan audit API dapat memberi bukti tambahan tentang penerapan aktual (CNCF GitOps Working Group, 2021). Riwayat tersebut tidak otomatis membuktikan siapa yang melakukan perubahan pada klaster atau apakah semua tindakan terekam; kelengkapan bergantung pada tata kelola repositori, identitas, konfigurasi audit, dan pengumpulan bukti. Penelitian ini karena itu menilai ketersediaan unsur bukti dengan rubrik, bukan menganggap salah satu sumber selalu lengkap.

2.6 *Runbook* sebagai Standarisasi Prosedur Pemulihan

Runbook adalah dokumen terstandarisasi yang berisi prosedur operasional untuk menanggapi insiden atau melakukan pemulihan (*recovery*) pada sebuah sistem.

Dalam ranah operasi TI, *runbook* berfungsi sebagai *playbook* yang mendefinisikan langkah-langkah korektif berurutan berdasarkan jenis insiden, sehingga setiap *operator* yang berbeda mengikuti urutan tindakan yang sama (Schlette et al., 2024). Penggunaan *runbook* mengurangi variabilitas akibat keahlian *operator* yang berbeda-beda dan memastikan bahwa keputusan subjektif diminimalkan selama pemulihan.

Dalam rancangan penelitian ini, prosedur imperatif bukan runbook yang dipandu kepada operator. Lingkungan A menggunakan pemeriksa dan korektor otomatis yang tetap, dengan pembandingan deklarasi dan keadaan aktual serta tindakan `kubectl` yang telah ditetapkan. Pilot perlu memverifikasi bahwa prosedur tersebut bekerja tanpa menerima identitas skenario atau label injeksi dan menyimpan bukti yang sama jenisnya dengan evaluator bersama.

2.7 Paradigma *Deployment* Alternatif

Pilihan pengelolaan konfigurasi mencakup rekonsiliasi berbasis tarik, pipeline dorong, alat konfigurasi yang dijalankan sesuai kebutuhan, serta otomasi imperatif khusus. Perbedaan autentikasi, cakupan sumber daya, pencatatan, dan frekuensi eksekusi bergantung pada konfigurasi implementasi; tidak diasumsikan bahwa seluruh pipeline dorong mengekspos kredensial atau bahwa semua alat imperatif tidak memiliki audit. Studi Shrestha dan Ali secara khusus membandingkan GitOps dengan Ansible pada waktu induksi, penemuan, dan pemulihan drift (Shrestha and Ali, 2024).

Penelitian ini membatasi pembanding pada satu prosedur imperatif terotomasi berbasis `kubectl` dan satu instalasi ArgoCD. Kontras ini tidak merupakan batas atas efek yang berlaku bagi semua alat atau konteks; temuan nantinya hanya berlaku bagi implementasi dan workload yang benar-benar diuji.

2.8 Evaluasi Empiris GitOps dan ArgoCD

Studi terkait telah menguji sejumlah kombinasi alat dan tujuan penelitian. Shrestha dan Ali (Shrestha and Ali, 2024) membandingkan GitOps dengan Ansible pada waktu induksi, penemuan, dan pemulihan drift. Kagganti Nataraja (Nataraja, 2025) mendeskripsikan perbandingan terkontrol pada klaster Minikube satu node dengan konfigurasi keamanan yang berbeda. Matubber (Matubber, 2025) mengevaluasi pengelolaan infrastruktur Kubernetes berbasis GitOps dan mencakup pembanding lama yang semi-terotomasi. Ketiganya menunjukkan bahwa penelitian komparatif atau

evaluasi berbasis waktu telah ada; abstrak yang tersedia tidak cukup untuk menyimpulkan ada atau tidaknya uji statistik tertentu.

Utami dan Al Fatta (Utami and Fatta, 2021) membahas model deployment deklaratif berbasis tarik dibandingkan dengan model berbasis dorong. Paavola (Paavola, 2021) membahas pengelolaan beberapa aplikasi Kubernetes menggunakan prinsip GitOps, sedangkan D'Amore (D'Amore, 2021) mendeskripsikan implementasi ArgoCD pada lingkungan Kubernetes hybrid cloud. Studi tersebut memberi konteks implementasi, tetapi tidak diperlakukan di sini sebagai bukti bahwa salah satu prosedur selalu lebih unggul.

Perbandingan penelitian perlu dibaca sesuai tujuan, alat, lingkungan, konfigurasi, dan hasil yang benar-benar dilaporkan oleh masing-masing sumber. Draf ini tidak menyatakan bahwa penelitian terdahulu tidak terkontrol, tidak menggunakan statistik, atau tidak memiliki baseline apabila pernyataan itu tidak dapat dipastikan dari abstrak yang tersedia.

2.9 Metode Eksperimental Terkontrol pada Riset Sistem

Evaluasi empiris pada riset sistem dan infrastruktur memerlukan desain eksperimental yang memitigasi ancaman terhadap validitas (Wohlin et al., 2012). Limoncelli et al. (2018) menjelaskan bahwa penelitian sistem harus membangun lingkungan yang dapat direproduksi, mengendalikan variabel pengganggu, dan mendefinisikan metrik yang jelas sebelum menyimpulkan hubungan kausal.

Rancangan penelitian ini menetapkan prosedur imperatif terotomasi berbasis `kubectl` dan prosedur GitOps berbasis ArgoCD sebagai dua perlakuan yang dibandingkan. Kesetaraan workload, nilai dasar, inventaris pengukuran, dan prosedur injeksi direncanakan untuk mengurangi perbedaan yang tidak relevan. Karena kedua implementasi dapat berbeda dalam lebih dari satu komponen teknis, hasil tidak langsung diatribusikan sebagai efek universal paradigma; interpretasinya dibatasi pada prosedur yang benar-benar diuji (Wohlin et al., 2012).

Analisis yang direncanakan bersifat per skenario dan eksploratif. Ringkasan distribusi, jumlah observasi lengkap dan tersensor, serta ukuran efek dilaporkan bersama-sama; uji komparatif hanya digunakan jika asumsi dan struktur data mendukungnya. Target pengulangan tidak dianggap sebagai bukti daya statistik yang memadai. Rincian definisi operasional dan batas interpretasi terdapat pada Bab III.

2.10 Klasifikasi Beban Uji (*Workload*)

Aplikasi yang dijalankan pada klaster Kubernetes dapat dikelompokkan ke dalam beberapa kelas *workload* berdasarkan karakteristik arsitektur dan manajemen *state* (Siddiqui et al., 2023; Aghili et al., 2024):

1. *Workload* web tanpa state (*stateless web microservice*): Layanan yang tidak menyimpan *state* secara internal, sehingga setiap permintaan dapat dilayani oleh instans mana pun. Contoh: server HTTP sederhana, *API gateway*.

2. *Workload* berbasis data yang menyimpan *state* (*stateful data service*): Layanan yang menyimpan *state* secara persisten, seperti *database* relasional, *search engine*, atau *object storage*. *Workload* kelas ini memerlukan urutan *deployment* dan penanganan penyimpangan yang lebih kompleks.

3. *Workload* komposit multi-service: Aplikasi yang terdiri atas banyak komponen yang saling bergantung, mencakup kombinasi kelas 1 dan 2 beserta komponen infrastruktur (*ingress*, TLS, *secrets*, *monitoring*, *backup*, kebijakan keamanan). Kelas *workload* ini memiliki kompleksitas dependensi tertinggi dan paling rentan terhadap penyimpangan konfigurasi akibat banyaknya komponen yang harus diselaraskan.

Pengujian formal dalam rancangan ini menggunakan beban uji komposit multi-service InvenioRDM. Inventaris dan komponen aktual harus dicatat dari konfigurasi yang divalidasi sebelum eksperimen; jumlah komponen yang pernah disebut sebagai target belum diverifikasi. Beban uji merupakan konteks pengukuran, bukan dasar untuk menggeneralisasi hasil ke seluruh workload multi-service. Surrogat sederhana yang digunakan dalam pilot tidak menggantikan atau memperluas sampel formal.

2.11 Ruang Kontribusi Penelitian

Studi komparatif mengenai GitOps dan pengelolaan konfigurasi telah ada, termasuk evaluasi terhadap Ansible, pengujian terkontrol pada Minikube, dan pengukuran yang mencakup baseline semi-terotomasi. Karena itu, penelitian ini tidak berangkat dari klaim bahwa perbandingan terdahulu tidak ada atau bahwa keunggulan GitOps telah terbukti secara universal.

Ruang kontribusi yang direncanakan lebih terbatas: mengevaluasi dua prosedur yang didefinisikan secara eksplisit, yaitu pemeriksa/korektor imperatif terotomasi berbasis `kubectl` dan rekonsiliasi ArgoCD, pada workload multi-service Invenio-RDM; menggunakan evaluator kesetaraan bidang yang sama; membedakan waktu deteksi, bukti penyimpangan, dan pemulihan; serta memperlakukan sumber daya di luar cakupan aplikasi sebagai kasus batas visibilitas tersendiri. Kebaruan relatif dan relevansi kontribusi ini masih perlu dikonfirmasi melalui peninjauan pustaka lengkap dan pembimbing.

Hasil yang kelak diperoleh hanya dapat menjawab pertanyaan pada skenario, workload, konfigurasi, dan resolusi instrumentasi yang benar-benar diuji. Besaran maupun arah perbedaan belum dapat dipastikan sebelum pengumpulan serta analisis data formal.

BAB III

METODE PENELITIAN

3.1 Deskripsi Umum dan Status Protokol

Penelitian ini dirancang sebagai evaluasi empiris terkontrol atas dua prosedur pengelolaan *deployment*: prosedur imperatif terotomasi berbasis `kubectl` dan GitOps berbasis ArgoCD. Pertanyaan yang dituju ialah bagaimana kedua prosedur tersebut berbeda pada konsistensi konfigurasi, waktu deteksi, ketersediaan bukti penyimpangan, pemulihan konfigurasi, dan keterlacakan untuk skenario yang ditetapkan. Estimand dibatasi pada selisih hasil antarprosedur dalam konteks beban uji multi-service InvenioRDM dan apparatus eksperimen yang kelak diverifikasi; estimand ini bukan efek universal seluruh paradigma imperatif atau GitOps.

Naskah ini memuat rancangan protokol, bukan laporan pelaksanaan. Protokol masih memerlukan tinjauan pembimbing, dan apparatus eksperimen yang tersedia belum divalidasi untuk mengimplementasikan seluruh definisi operasional di bawah. Versi perangkat lunak, kesiapan klaster, inventaris sumber daya formal, dan data eksperimen belum ditetapkan atau diverifikasi dalam naskah ini. Karena itu, uraian rencana tidak boleh dibaca sebagai bukti bahwa eksperimen telah dilaksanakan.

Penyesuaian kepemilikan pengendali tetap berstatus draf untuk tinjauan pemilik penelitian dan pembimbing: HPA normal aktif pada kedua lengan dan skenario A menjadi kontrol respons HPA. A tidak digabungkan ke efikasi ArgoCD pada B–E/G; F tetap kasus batas visibilitas tersendiri. Seluruh eksekusi formal dinonaktifkan. Instrumen saat ini memvalidasi bukti tersimpan secara offline, bukan menjalankan injeksi, deployment atau reset langsung; adapter polling, injeksi, pemulihan, audit dan SMTP belum selesai.

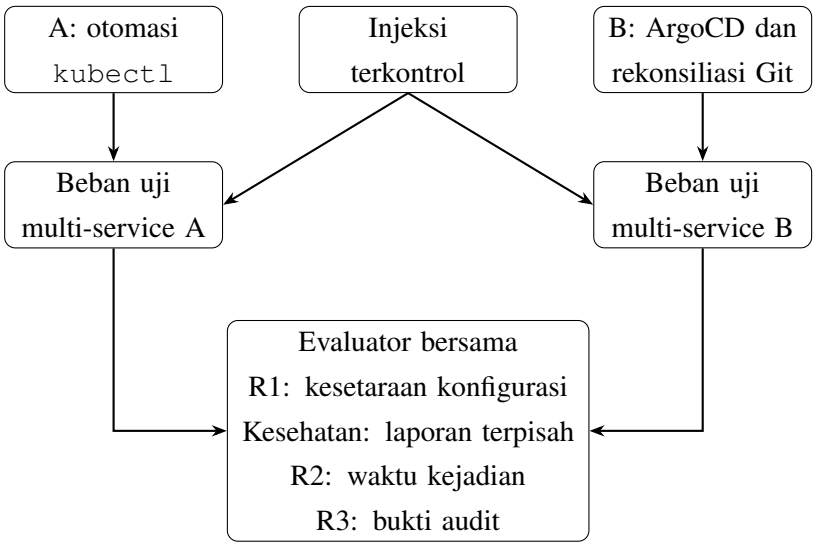
3.2 Rancangan Perbandingan

Pengendalian kondisi awal, pengulangan, pencatatan penyimpangan protokol, dan pembatasan inferensi mengikuti prinsip evaluasi empiris perangkat lunak (Wohlin et al., 2012). Karena pengujian memakai lingkungan yang dibangun secara sengaja pada satu klaster, rancangan tidak dianggap sebagai penugasan acak klaster dari suatu populasi (Cook and Campbell, 1979).

Unit analisis adalah satu injeksi penyimpangan yang diterima pada satu ling-

kungan dalam satu pengulangan. Lingkungan A menggunakan otomasi imperatif tetap: pemeriksa berkala membandingkan keadaan aktual dengan inventaris dan nilai yang diinginkan, menyimpan perbedaan yang teramati, lalu menjalankan prosedur koreksi `kubectl` yang telah ditentukan. Lingkungan B menggunakan ArgoCD dengan rekonsiliasi otomatis terhadap deklarasi Git. Tidak ada operator yang menunggu, mengetik, atau memilih tindakan korektif selama pengukuran; waktu respons manusia maupun tingkat keahlian operator bukan variabel penelitian.

Kedua lingkungan memakai beban uji, nilai dasar, definisi bidang yang diperiksa, serta prosedur injeksi dan pengukuran yang setara. Injeksi dilakukan satu per satu, bukan serentak, dan urutan lingkungan serta skenario diacak dalam blok pengujian setelah kondisi dasar dipulihkan. Pengendali koreksi hanya menerima keadaan aktual dan deklarasi yang diperlukan untuk membandingkannya; identitas skenario tidak diteruskan sebagai label, termasuk pada skenario Secret. Rancangan ini membandingkan dua implementasi operasional yang telah ditentukan, bukan mengisolasi seluruh faktor yang mungkin membedakan setiap produk atau organisasi.

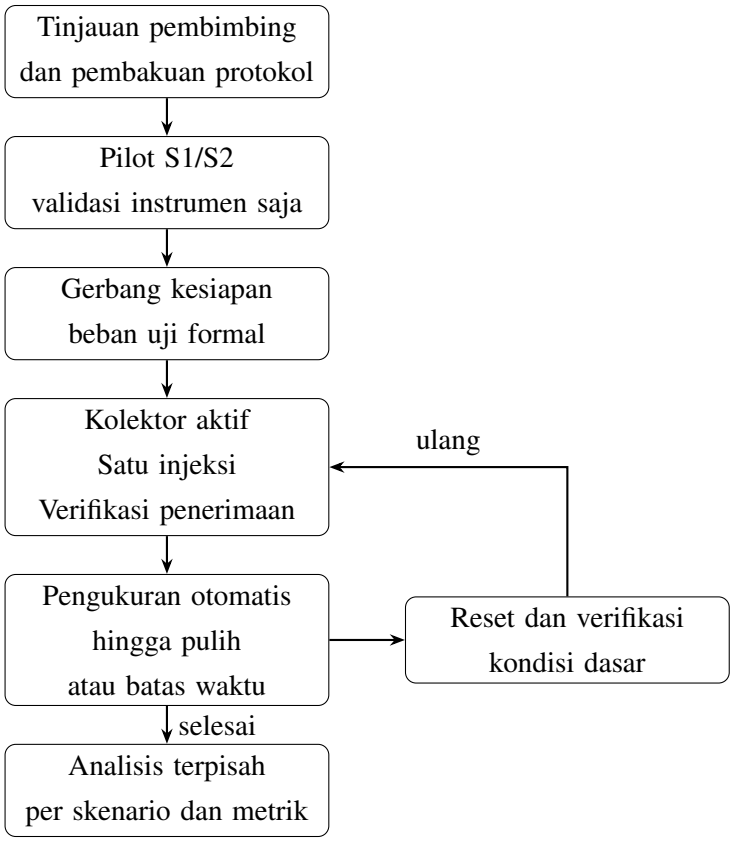


Gambar 3.1: Rancangan perbandingan dua prosedur otomatis dengan evaluator dan definisi metrik yang sama.

Gambar 3.1 menunjukkan bahwa injeksi, mekanisme pengelolaan, dan evaluator merupakan bagian yang dibedakan secara eksplisit. Hasil hanya akan ditafsirkan untuk konfigurasi apparatus yang benar-benar dicatat saat eksperimen.

3.3 Alur Penelitian

Tahapan yang direncanakan meliputi peninjauan protokol, validasi instrumen melalui uji pendahuluan, pengujian formal, dan analisis per skenario. Uji pendahuluan menggunakan surrogat sederhana S1 dan S2 hanya untuk memeriksa kelayakan injeksi, pencatatan waktu, pemeriksa kesetaraan, dan prosedur reset. Data S1/S2 tidak masuk ke sampel formal dan tidak digunakan untuk menyimpulkan perilaku InvenioRDM.



Gambar 3.2: Bagan alir protokol, dengan pilot dipisahkan dari pengulangan formal.

Gambar 3.2 menempatkan validasi sebelum pengumpulan data formal dan mensyaratkan pemulihan kondisi dasar di antara pengulangan. Tidak ada jeda buatan setelah injeksi untuk menunggu respons manusia.

3.4 Beban Uji dan Lingkungan

Pengujian formal direncanakan menggunakan beban uji komposit multi-service InvenioRDM dengan sasaran sedikitnya 16 komponen aplikasi dan infrastruktur pendukung sesuai ruang lingkup proposal. Sasaran tersebut bukan jumlah komponen yang telah diverifikasi. Inventaris formal mencakup seluruh sumber daya dalam manifest yang disetujui untuk kedua lingkungan; daftar komponen, batas pengelolaan, dan versinya akan dicatat dari artefak yang benar-benar digunakan. Komponen infrastruktur bersama harus dibedakan dari sumber daya yang menjadi sasaran injeksi agar isolasi kedua lingkungan dapat diperiksa. S1/S2 merupakan surrogat pilot, bukan pengganti beban formal, dan tidak boleh digabungkan ke hasil atau analisis formal.

Rancangan membatasi pengujian pada satu klaster Kubernetes dan satu implementasi GitOps, ArgoCD. Spesifikasi serta versi aktual klaster, node, komponen beban uji, dan ArgoCD harus diverifikasi dan dicatat sebelum pengujian; nilai yang pernah disebut sebagai sasaran pada draf sebelumnya belum menjadi fakta eksperimen. Dua lingkungan perlu memiliki inventaris dan nilai dasar yang sebanding serta terisolasi agar koreksi satu lingkungan tidak mengubah lingkungan lain.

Konteks deployment yang dilaporkan pemilik adalah Kubernetes 1.32.6/RKE2, containerd, dan Ubuntu 24.04, berbeda dari target lama proposal. Penunjuk sumber otoritatif ada pada `experiments/deployment-source.json`, dengan revisi referensi `6cacea6642a7c1690243d3b54130fa08216a9595`. Application `argocd/apps/invenio-bootstrap.yaml` memilih `k8s/apps/invenio/kustomization.yaml`; konfigurasi sumber rekonsiliasi menetapkan 30 detik, bukan target lama 60 detik. Revisi referensi bukan bukti revisi yang diterapkan. Sebelum formal, rekam revisi runtime aktual, render dan digest-nya, image digest, inventaris serta interval pengendali; tidak dibuat sumber manifest editable kedua di repositori penelitian.

3.4.1 Gerbang Kesiapan

Pengulangan formal hanya dimulai setelah seluruh prasyarat berikut diperiksa dan dicatat: (1) manifest dan inventaris sumber daya formal telah dibekukan; (2) kedua lingkungan berada pada kesetaraan nilai yang ditetapkan dan kesehatan layanan telah memenuhi kriteria kesiapan; (3) auto-sync, self-heal, cakupan aplikasi, dan ke-

bijakan prune ArgoCD telah diverifikasi sesuai rancangan; (4) pemeriksa imperatif, pengumpul bukti, serta pembanding bidang telah lulus uji pilot; (5) jam pencatat waktu tersinkronisasi; dan (6) tidak ada pengujian atau perubahan lain yang berlangsung pada lingkup sumber daya yang diamati. Ambang kesehatan, batas beban, interval ArgoCD, serta batas waktu pengamatan akan dibakukan setelah pilot dan tinjauan pembimbing; naskah ini tidak menyatakan nilai aktualnya.

Jika prasyarat gagal, injeksi tidak dimulai. Gangguan setelah injeksi dicatat sebagai kejadian pengujian, bukan dihapus tanpa jejak. Kriteria penghentian, nilai batas waktu, dan aturan pengulangan harus ditetapkan sebelum data formal dikumpulkan.

Gerbang CPU dan memori tetap ketat di bawah 70% pada setiap node; memori memakai working set metrics-server dibagi allocatable. Bukti metrik, inventaris, resource requests/limits, pod Running dan Ready, audit, SMTP serta reset aplikasi wajib lengkap. Kondisi worker02 sekitar 71% yang dilaporkan pemilik, audit belum tersedia, SMTP placeholder, kapasitas dua lingkungan serta private-draft seed/recovery yang tertunda tidak dianggap teratasi. Bootstrap/migrasi/fixtures/indexing/queue tidak boleh dipicu sebagai bagian repair rutin tanpa desain lifecycle yang ditinjau. Interval rekonsiliasi repositori, watch kejadian, retry self-heal dan evaluator dibedakan.

3.5 Intervensi dan Skenario

Intervensi adalah satu perubahan terkontrol terhadap keadaan sumber daya dalam satu pengulangan. Skenario A–E dan G menyasar sumber daya yang termasuk inventaris aplikasi; skenario F menguji batas visibilitas untuk sumber daya di luar cakupan aplikasi dan dianalisis terpisah.

Tabel 3.1: Skenario intervensi yang direncanakan

Skenario	Intervensi	Batas interpretasi
A	Mengubah jumlah replika Deployment yang dikendalikan HPA	Kontrol HPA normal pada kedua lengan; bukan koreksi ArgoCD atas replika yang diabaikan
B	Mengubah referensi image Deployment terkelola	Perubahan bidang yang telah ditetapkan dalam skema pengukuran
C	Mengubah bidang ConfigMap terkelola	Tidak mengubah deklarasi sumber kebenaran
D	Menghapus sumber daya terkelola	Pemulihan dinilai terhadap inventaris dan nilai dasar
E	Mengubah bidang sumber daya terkelola melalui patch	Hanya bidang yang telah ditetapkan yang dinilai
F	Membuat atau menerapkan sumber daya pada namespace di luar cakupan aplikasi	Kasus batas visibilitas sumber daya tak terkelola; bukan keberhasilan atau kegagalan self-heal yang diasumsikan
G	Mengubah data Secret terkelola di luar deklarasi	Perbandingan dilakukan tanpa menyimpan atau menampilkan nilai Secret

Pada skenario A, `/spec/replicas` web/worker dimiliki HPA dan dikecualikan dari kesetaraan statis. Respons diukur terhadap keputusan HPA yang teramati, bukan angka tetap tiga replika. Tidak adanya OutOfSync tidak berarti deteksi nol; HPA dapat memulihkan replika tanpa sinyal ArgoCD. Hasil A dilaporkan sebagai kontrol HPA terpisah. Penghormatan kepemilikan replika saat sync harus diverifikasi, bukan diasumsikan dari ignoreDifferences saja.

Pada skenario F, sumber daya di luar inventaris aplikasi tidak dianggap sebagai drift terkelola yang wajib dipulihkan ArgoCD. Pengamatan dibatasi pada apakah batas cakupan dan pencatatan inventaris membuat keberadaan sumber daya tersebut terlihat. A dan F tidak digabungkan ke estimasi utama B–E dan G. Pada G, Secret sin-

tetis harus diisolasi dari rekonsiliasi SealedSecret sebelum koreksi dapat diatribusikan kepada prosedur yang dibandingkan; pembekuan ImageUpdater juga memerlukan verifikasi.

3.6 Definisi Variabel dan Metrik

3.6.1 R1: Kesetaraan Konfigurasi dan Kesehatan

Kesetaraan konfigurasi dinilai menggunakan aturan yang sama pada kedua lingkungan. Sebelum eksperimen, skema bidang yang diperiksa dan inventaris sumber daya yang diharapkan dibekukan. Bidang dinamis yang tidak relevan bagi deklarasi dikecualikan secara eksplisit. Untuk setiap sumber daya yang diharapkan, sumber daya dinyatakan cocok jika keberadaannya sesuai inventaris dan semua bidang terpilih sama dengan nilai yang diinginkan.

$$R1_{eq} = \frac{\text{jumlah sumber daya terkelola yang cocok}}{\text{jumlah sumber daya dalam inventaris yang diharapkan}} \times 100\%.$$

Evaluator menghitung $R1_{eq}$ dari pembacaan keadaan aktual dan inventaris yang sama bentuknya untuk kedua lengan; status *Synced* ArgoCD bukan pengganti aturan kesetaraan tersebut. Nilai dicatat pada kondisi dasar, pengamatan pertama setelah injeksi, dan setelah prosedur koreksi. Jika koreksi mendahului pengamatan pascainjeksi, kondisi sesaat tersebut ditandai tidak teramati; kondisi drift tidak direkonstruksi seolah-olah telah diukur. Nilai Secret dibandingkan secara aman di memori; nilai mentah maupun hash yang dapat digunakan untuk menebak isinya tidak ditulis ke log, gambar, atau laporan.

Kontrak bidang mencakup image, requests/limits, ConfigMap, Secret dan bidang Service yang relevan. Bidang dinamis HPA tidak dibandingkan dengan nilai replika statis Git. Bidang yang tidak diperiksa atau kegagalan API menggagalkan bukti; sumber daya yang benar-benar tidak ada pada daftar lengkap memberi ketidakcocokan. Kesehatan rollout tidak menggantikan pemeriksaan bidang tersebut.

Kesehatan layanan dilaporkan sebagai hasil terpisah berdasarkan kriteria kesiapan yang ditetapkan sebelum eksperimen, misalnya kondisi kesiapan Pod dan status layanan yang memang tersedia pada workload. Kesehatan tidak dimasukkan ke pembilang kesetaraan konfigurasi. Sumber daya ekstra di luar inventaris, termasuk F, dilaporkan terpisah dan tidak mengubah penyebut R1.

3.6.2 R2: Deteksi, Ketersediaan Bukti, dan Pemulihan

Semua waktu berasal dari satu kolektor pada host pencatat. Setiap kejadian diberi waktu jam monotonik dalam detik dengan pecahan desimal serta cap waktu UTC untuk audit; durasi dihitung dari jam monotonik, bukan dari selisih jam dinding. Pengamatan diaktifkan sebelum injeksi dan berlangsung pada interval nominal satu detik. Cap waktu adalah waktu pengamatan pertama, bukan waktu kejadian yang diinterpolasi; interval aktual dan jeda polling dicatat. Interval ini merupakan resolusi pengamatan yang direncanakan dan harus divalidasi pada pilot. Interval pemeriksa imperatif dan jadwal rekonsiliasi ArgoCD dicatat tersendiri dari interval evaluator, dibekukan sebelum pengujian formal, dan tidak disesuaikan setelah hasil terlihat.

- t_0: waktu kolektor mulai mengirim permintaan injeksi ke Kubernetes API. Waktu penerimaan respons API juga dicatat sebagai t_a, sehingga $t_a - t_0$ mendokumentasikan latensi permintaan. Injeksi diterima hanya jika operasi API berhasil dan objek respons atau pembacaan balik memverifikasi perubahan bidang sasaran. Durasi sejak t_0 mencakup latensi permintaan, bukan semata-mata waktu respons setelah perubahan tersimpan. Injeksi gagal atau tidak terverifikasi dicatat sebagai kegagalan injeksi, bukan sebagai waktu deteksi nol.

- t_d: waktu pertama mekanisme pengelolaan memberi sinyal deteksi berdasarkan perbedaan dari nilai yang diinginkan. Pada A, sinyal berasal dari pemeriksa imperatif; pada B, dari status perbedaan ArgoCD yang relevan. Tidak ada sinyal skenario tersembunyi yang diberikan kepada mekanisme tersebut.

- t_e: waktu pertama bukti dari mekanisme pengelolaan tersimpan dan mengidentifikasi sumber daya serta bidang yang berbeda terhadap acuan. Bukti harus dapat diperiksa ulang tanpa memakai label injeksi yang tersembunyi; untuk Secret cukup dicatat identitas bidang dan hasil ketidakcocokan, bukan isinya. Kriteria ini hanya mengukur bukti deviasi, bukan kelengkapan seluruh unsur R3 atau identifikasi penyebab. t_e dapat sama dengan t_d, tetapi keduanya tetap dicatat terpisah. Pengumpulan bukti tidak menunda koreksi pada salah satu lingkungan.

- t_r: waktu pengamatan pertama ketika inventaris dan seluruh bidang R1 kembali cocok dengan nilai yang diinginkan, dengan kecocokan dikonfirmasi pada pembacaan berikutnya. Waktu yang dilaporkan tetap waktu pengamatan perta-

ma; konfirmasi hanya mencegah keadaan sesaat dinyatakan pulih. Kesehatan layanan dilaporkan terpisah.

Durasi utama adalah waktu deteksi $t_d - t_0$, waktu ketersediaan bukti $t_e - t_0$, dan waktu pemulihan konfigurasi penuh $t_r - t_0$. Selang dari deteksi ke bukti ($t_e - t_d$) dapat ditampilkan sebagai rincian tambahan. t_r berarti pemulihan kesetaraan inventaris dan bidang yang diukur, bukan klaim bahwa seluruh layanan sehat atau tidak mengalami dampak lain.

Sinyal deteksi A berasal dari pemeriksaan otomatis terhadap keadaan aktual; setelah sinyal muncul, prosedur koreksi tetap menerapkan deklarasi yang diinginkan menggunakan tindakan `kubectl` yang telah ditetapkan. B menggunakan rekonsiliasi ArgoCD. Kedua prosedur berjalan otomatis tanpa waktu reaksi manusia dan tanpa jeda buatan. Keputusan koreksi menggunakan hasil perbandingan umum, bukan label A–G yang disembunyikan dari responden.

3.6.3 R3: Rubrik Bukti Keterlacakan

R3 dinilai deskriptif untuk setiap pengulangan dengan rubrik yang sama, bukan diasumsikan lengkap karena salah satu alat digunakan. Penilai mencatat ada, sebagian, atau tidak ada bukti untuk unsur berikut.

Tabel 3.2: Rubrik penilaian bukti R3

Unsur	Kriteria bukti	Contoh sumber yang diperiksa
Pelaku	Identitas aktor atau akun layanan yang melakukan perubahan dapat ditelusuri	Log audit API Kubernetes; log eksekusi otomatis
Objek dan perubahan	Objek serta bidang yang berubah dapat ditentukan tanpa menampilkan data sensitif	Diff ArgoCD yang telah disanitasi; catatan perbandingan imperatif
Waktu dan urutan	Waktu serta urutan injeksi, deteksi, dan pemulihan dapat direkonstruksi	Cap waktu kolektor; log audit; metadata Git
Tindakan dan hasil	Tindakan koreksi dan hasil verifikasinya dapat dikaitkan dengan pengulangan	Log koreksi; riwayat sinkronisasi; pembacaan verifikasi

Rubrik ini mengukur ketersediaan bukti perubahan dan tindakan. Rubrik tidak

meminta penilai menyimpulkan penyebab kausal atau akar masalah dari sebuah diff. Data audit dapat memuat identitas sensitif; pelaporan hanya menggunakan informasi minimum yang telah disanitasi.

3.7 Protokol Pengulangan dan Penanganan Kegagalan

Satu pengulangan mengikuti urutan berikut:

1. Periksa gerbang kesiapan dan simpan bukti kondisi dasar, inventaris, kesehatan, serta status instrumentasi.

2. Aktifkan koleksi sinyal deteksi, bukti, kesetaraan, dan kesehatan sebelum mengirim injeksi; pastikan mekanisme pengelolaan juga telah aktif. Jalankan hanya satu skenario pada satu lingkungan. Catat t_0, respons API pada t_a, dan verifikasi penerimaan injeksi tanpa menambahkan label atau identitas skenario ke sumber daya yang diamati.

3. Pertahankan koleksi berkala satu detik tanpa menyisipkan waktu tunggu agar manusia bereaksi atau agar drift bertahan selama durasi tertentu. Jika perubahan diterima tetapi sinyal singkat tidak teramati, laporkan keterbatasan observasi tanpa menganggap deteksi terjadi pada waktu nol.

4. Lanjutkan sampai pemulihan terkonfirmasi atau batas waktu pengamatan yang telah dibakukan tercapai. Simpan semua kejadian, jeda polling, kesalahan API, serta status penghentian.

5. Pulihkan kondisi dasar, verifikasi seluruh bidang/inventaris, kesehatan, HPA, tidak adanya objek karantina, adopsi runtime serta keadaan aplikasi/data/queue; baru mulai pengulangan berikutnya. Kegagalan tindakan koreksi diteruskan, bukan ditelan. Tidak ada izin purge, restore atau modifikasi data dari protokol ini.

Batas waktu maksimum pengamatan harus ditentukan dan dibekukan setelah pilot serta tinjauan pembimbing, sebelum pengumpulan formal. Tidak ada nilai batas waktu yang diklaim telah diuji dalam draf ini. Jika sinyal deteksi, bukti, atau pemulihan belum terjadi saat batas itu tercapai, hasil masing-masing ditandai tersensor kanan pada batas tersebut; waktu hilang tidak diganti nol dan pengulangan tidak dikeluarkan hanya karena pemulihannya gagal. Kegagalan injeksi, kegagalan instrumentasi,

gangguan infrastruktur, dan tidak terjadinya deteksi merupakan kategori berbeda. Setiap kategori, jumlah pengulangan terdampak, alasan, serta aturan pengulangan yang telah disetujui dilaporkan.

Run dengan respons injeksi sukses tetapi tanpa bukti bahwa perubahan bidang sasaran pernah diterima tidak menjadi hasil nol; run ditandai sebagai injeksi tidak valid dan dicatat. Jika perubahan diterima tetapi mekanisme tidak mendeteksinya sampai batas pengamatan, itu merupakan hasil deteksi tersensor, bukan kegagalan instrumen. Jika kolektor atau sumber data gagal sehingga status tidak dapat ditentukan, run ditandai kegagalan instrumentasi dan tidak ditafsirkan sebagai deteksi terlewat. Skenario F tetap diklasifikasikan sebagai pengamatan batas visibilitas dan tidak dinilai dengan asumsi pemulihan otomatis.

3.8 Pengulangan dan Analisis Data

Target rancangan adalah $n = 5$ pengulangan per skenario dan per lingkungan untuk R1 serta $n = 10$ untuk R2. Angka tersebut merupakan target deskriptif yang direncanakan, bukan hasil analisis daya dan bukan jaminan kecukupan daya untuk setiap skenario. Pengulangan R3 mengikuti pengumpulan bukti pada run yang sama dan dianalisis secara deskriptif. S1/S2 tidak dihitung dalam target ini. A sebagai kontrol HPA dan F sebagai batas visibilitas dilaporkan tersendiri, tidak dimasukkan ke efikasi drift terkelola B–E dan G.

Hasil disajikan per skenario dan per lingkungan: jumlah run direncanakan, jumlah injeksi valid, jumlah kejadian teramati dan tersensor, ringkasan distribusi waktu, proporsi pemulihan pada batas pengamatan, R1, kesehatan terpisah, serta rubrik R3. Nilai mentah yang hilang tidak diimputasi sebagai nol. Observasi yang tersensor tidak diam-diam dibuang agar hanya run yang berhasil pulih yang dibandingkan. Jika penyensoran terjadi, ringkasan dan uji yang tidak memperhitungkan penyensoran tidak ditafsirkan sebagai perbandingan lengkap; analisis komparatif untuk hasil tersebut harus ditetapkan sebelum melihat hasil, atau dilaporkan secara deskriptif tanpa klaim inferensial.

Untuk hasil waktu yang teramati lengkap, rencana analisis eksploratif menggunakan uji Mann–Whitney U dua sisi apabila asumsi independensi pengulangan terpenuhi. Cliff's δ ditetapkan sebagai ukuran efek utama agar pilihan ukuran efek tidak bergantung pada arah hasil yang terlihat. Distribusi, median, rentang, dan ukuran sampel yang benar-benar tersedia tetap dilaporkan. Jika nilai p digunakan, ambang

nominalnya adalah $\alpha = 0{,}05$ dengan penyesuaian Holm untuk keluarga perbandingan waktu yang dibakukan sebelum pengumpulan formal. Definisi keluarga uji dan pemenuhan independensi harus ditinjau bersama pembimbing sebelum protokol dibekukan; tidak ada klaim konfirmatoris dari draf ini. Penggabungan lintas skenario tidak menjadi analisis utama karena bentuk dan dampak intervensinya berbeda. R1, kesehatan, R3, dan kasus F dilaporkan deskriptif; tidak ada hasil signifikansi atau besar efek yang dinyatakan sebelum data dikumpulkan.

3.9 Ancaman terhadap Validitas dan Keterbatasan Rancangan

Validitas internal dapat terancam oleh perbedaan beban klaster dari waktu ke waktu, perubahan konfigurasi selain intervensi, ketidaksetaraan inventaris, serta perbedaan jadwal rekonsiliasi. Isolasi namespace, nilai dasar yang dibekukan, urutan pengujian yang diacak, pencatatan keadaan, dan gerbang kesiapan dirancang untuk mengurangi ancaman tersebut, tetapi tidak menjamin bahwa satu-satunya perbedaan adalah paradigma abstrak. Perbandingan tetap mengikutsertakan perbedaan implementasi yang melekat pada pemeriksa/korektor `kubectl` dan ArgoCD.

Validitas konstruk dibatasi oleh tujuh skenario yang dipilih, definisi bidang yang dibandingkan, interval polling, batas waktu, kriteria kesehatan, dan rubrik bukti. Pengamatan polling memberikan waktu deteksi pada resolusi instrumen, bukan waktu kejadian kontinu yang persis. Skenario F menguji batas inventaris dan tidak dapat digeneralisasi sebagai pengukuran self-healing untuk sumber daya terkelola.

Validitas eksternal dibatasi pada satu klaster, satu beban uji formal multi-service InvenioRDM, satu alat GitOps, serta konfigurasi dan versi yang kelak benar-benar digunakan. Pilot S1/S2 tidak memperluas cakupan inferensi. Tidak ada kesimpulan yang dapat digeneralisasi ke semua aplikasi, klaster, atau alat GitOps dari rancangan ini saja.

Reliabilitas bergantung pada konsistensi kolektor, sinkronisasi jam, cakupan log, dan kemampuan mengulang reset. Sebelum data formal dikumpulkan, keberhasilan pencatatan, pembandingan Secret tanpa kebocoran, deteksi kegagalan, serta aturan pemulihan dan penyensoran harus diuji. Sampai gerbang tersebut dipenuhi, hasil eksperimen belum tersedia dan protokol tetap berstatus draf.

BAB IV
HASIL DAN ANALISIS

4.1 Status Data dan Provenans

Pada draf ini belum tersedia dataset eksperimen formal yang tervalidasi. Apparatus belum diuji terhadap keseluruhan protokol pada Bab III dan protokol masih menunggu tinjauan pembimbing. Oleh karena itu, bagian ini mendefinisikan informasi yang perlu dilaporkan, tetapi tidak menyajikan angka, uji statistik, ukuran efek, atau kesimpulan komparatif.

Sebelum pengisian hasil, dokumentasikan konfigurasi klaster yang benar-benar digunakan, versi Kubernetes dan ArgoCD, cakupan aplikasi, kebijakan auto-sync/self-heal/prune, manifest serta inventaris formal, interval polling aktual, sinkronisasi jam, dan kriteria kesiapan. Nilai yang pernah disebut sebagai target pada draf sebelumnya bukan hasil verifikasi. Laporkan pula setiap penyimpangan protokol, kegagalan kesiapan, dan perubahan apparatus.

Pengujian formal direncanakan pada workload multi-service InvenioRDM. Pilot S1/S2 hanya dapat mendukung laporan validasi instrumen dan prosedur; hasilnya tidak dipool atau ditafsirkan sebagai hasil formal. Identitas sensitif dan seluruh nilai Secret harus disanitasi dari log, tabel, gambar, serta lampiran.

4.2 Format Pelaporan E1: Kesetaraan dan Kesehatan (R1)

Kesetaraan inventaris dan bidang yang ditetapkan dilaporkan terpisah dari kesehatan layanan. Evaluator yang sama digunakan pada kedua lingkungan. Tabel 4.1 merupakan skema pelaporan, bukan hasil; isinya menunggu pengumpulan data formal.

Tabel 4.1: Skema hasil R1 per skenario dan lingkungan

Skenario	Lingkungan	Kesetaraan konfigurasi: dasar / pascainjeksi / pascapemulihan	Kesehatan layanan pada pengamatan terkait
B–E, G	kubectl / ArgoCD	Belum tersedia	Belum tersedia
A	HPA / HPA	Kontrol HPA; replika bukan kesetaraan statis	Belum tersedia
F	kubectl / ArgoCD	Tidak termasuk hasil utama R1; dicatat sebagai batas cakupan	Dilaporkan terpisah jika relevan

4.3 Format Pelaporan E2: Waktu (R2)

Untuk setiap skenario dan lingkungan, hasil kelak mencantumkan target run, jumlah injeksi terverifikasi, jumlah waktu teramati dan tersensor, serta durasi deteksi ($t_d - t_0$), ketersediaan bukti ($t_e - t_0$), dan pemulihan konfigurasi penuh ($t_r - t_0$). Nilai waktu dihitung dari cap waktu kolektor dan tidak diinterpolasi melewati jeda polling.

Tabel 4.2: Skema hasil R2 per skenario dan lingkungan

Skenario	Lingkungan	Run valid / tersensor dan alasan	Distribusi durasi: deteksi / bukti / pemulihan
B–E, G	kubectl / ArgoCD	Belum tersedia	Belum tersedia
A	HPA / HPA	Dilaporkan terpisah	Respons HPA, bukan efikasi ArgoCD
F	kubectl / ArgoCD	Dilaporkan sebagai pengamatan batas visibilitas	Tidak menjadi hasil pemulihan drift terkelola

Run tanpa deteksi, bukti, atau pemulihan sampai batas pengamatan akan tetap tampak sebagai tersensor kanan dan disertai jumlahnya. Kegagalan injeksi dan kegagalan instrumentasi dicatat terpisah dari kejadian mekanisme tidak mendeteksi. Run dengan pemulihan tidak lengkap tidak dihilangkan hanya untuk membentuk distribusi waktu yang lebih menguntungkan.

4.4 Format Pelaporan E3: Keterlacakan (R3)

Penilaian R3 akan disajikan berdasarkan rubrik yang sama untuk pelaku, objek/bidang perubahan, waktu/urutan, dan tindakan/hasil pemulihan. Setiap unsur diberi status ada, sebagian, atau tidak ada dengan rujukan ke bukti yang telah disanitasi. Tidak ada nilai hasil yang tersedia pada draf ini. Rubrik tidak menghasilkan waktu identifikasi akar masalah dan tidak digunakan untuk membuat klaim tentang penyebab kausal.

4.5 Rencana Analisis Komparatif

Ringkasan dan perbandingan dilakukan per skenario, bukan dengan menggabungkan semua intervensi menjadi satu angka. Target pengulangan R1 adalah lima run per skenario dan lingkungan, sedangkan R2 sepuluh run; keduanya merupakan target deskriptif dan belum dibuktikan cukup untuk mendeteksi efek pada setiap skenario. Statistik deskriptif, jumlah run valid, kejadian tersensor, serta ukuran efek yang sesuai harus dilaporkan bersama hasil.

Uji Mann–Whitney U hanya dapat dipertimbangkan secara eksploratif untuk hasil waktu yang teramati lengkap jika unit analisis dan asumsi independensi terpenuhi. Jika terdapat penyensoran, hasil uji yang mengabaikannya tidak boleh ditafsirkan sebagai perbandingan lengkap; metode untuk data tersensor harus dibakukan sebelum melihat hasil atau hasil dilaporkan deskriptif. Tidak ada nilai p, ukuran efek, ataupun interpretasi arah perbedaan yang dapat dituliskan sebelum pengumpulan serta analisis data formal.

4.6 Batas Interpretasi

RQ1 hanya dapat dijawab terhadap inventaris dan bidang yang ditetapkan; kesetaraan bukan bukti bahwa konfigurasi benar secara semantik. RQ2 membedakan sinyal deteksi, tersedianya bukti yang dapat diverifikasi, dan kembalinya konfigurasi ke keadaan yang diinginkan; kesehatan layanan tetap dilaporkan sebagai dimensi terpisah. RQ3 mendeskripsikan bukti yang tersedia, bukan akar masalah.

Skenario A mempertahankan HPA pada kedua lengan; respons replika bukan keberhasilan ArgoCD memulihkan bidang yang diabaikan. F merupakan pengujian batas visibilitas sumber daya tak terkelola, bukan janji prune/self-heal. A dan F dilaporkan tersendiri, tidak digabungkan ke efikasi B–E dan G. Tidak adanya OutOfSync

bukan deteksi nol.

Sampai data formal tersedia, tidak dapat dinyatakan prosedur mana yang lebih konsisten, lebih cepat, atau lebih terlacak. Kondisi ini merupakan batas bukti draf, bukan hasil eksperimen negatif dan bukan bukti bahwa instrumentasi gagal.

BAB V
KESIMPULAN DAN SARAN

5.1 Kesimpulan

Naskah ini telah merumuskan rancangan untuk membandingkan prosedur imperatif terotomasi berbasis `kubectl` dengan GitOps berbasis ArgoCD pada workload formal multi-service InvenioRDM. Rancangan membedakan kesetaraan konfigurasi dari kesehatan layanan, memisahkan waktu deteksi, ketersediaan bukti, dan pemulihan konfigurasi penuh, serta menggunakan rubrik deskriptif untuk keterlacakan.

Kesimpulan empiris untuk setiap pertanyaan penelitian belum dapat ditarik:

1. **RQ1:** Perbedaan kesetaraan konfigurasi dan kesehatan layanan belum diketahui karena data formal belum tersedia.
2. **RQ2:** Perbedaan waktu deteksi, ketersediaan bukti penyimpangan, dan pemulihan konfigurasi belum diketahui. Tidak ada hasil waktu yang dapat diisi dengan asumsi atau angka sintetik.
3. **RQ3:** Perbandingan kelengkapan bukti keterlacakan belum dapat dinilai sebelum log dan artefak formal diperiksa.

Dengan demikian, draf ini tidak menyimpulkan efektivitas, keunggulan, atau kelayakan adopsi salah satu prosedur. Skenario F kelak harus dilaporkan terpisah sebagai batas visibilitas sumber daya tak terkelola, bukan sebagai bukti keberhasilan atau kegagalan rekonsiliasi.

5.2 Saran

Sebelum pengujian formal, protokol perlu ditinjau pembimbing, lalu apparatus harus divalidasi terhadap definisi penerimaan injeksi, skema inventaris dan bidang, keselamatan pencatatan Secret, interval polling, timestamp, kriteria kesehatan, batas waktu, serta mekanisme reset. Pilot S1/S2 hanya digunakan untuk memeriksa instrumen; setelah pilot, konfigurasi formal multi-service InvenioRDM tetap harus diuji sesuai rancangan dan tidak boleh digantikan oleh surrogat.

Pengumpulan data berikutnya sebaiknya mencatat konfigurasi serta versi yang benar-benar digunakan, mempertahankan semua kategori kegagalan dan penyensoran, melaporkan hasil per skenario, dan menyimpan bukti yang telah disanitasi. Klaim komparatif perlu dibatasi pada prosedur, workload, klaster, dan resolusi instrumen yang diuji. Penelitian lanjutan dapat menguji workload, ukuran klaster, atau implementasi GitOps lain, tetapi perlu membakukan definisi metrik dan membedakan setiap pembanding secara eksplisit.

5.3 Keterbatasan Penelitian

Keterbatasan yang telah melekat pada rancangan ini meliputi penggunaan satu klaster, satu workload formal, dan satu implementasi GitOps; hasilnya tidak otomatis berlaku bagi konteks lain. Prosedur imperatif yang diuji adalah satu otomasi `kubectl` tertentu, sehingga tidak mewakili semua cara kerja imperatif. Interval polling membatasi ketelitian waktu kejadian, dan target pengulangan bukan jaminan daya statistik per skenario.

Skenario F berada di luar cakupan sumber daya terkelola dan hanya menguji batas visibilitas. Pengukuran kesetaraan tidak menilai apakah konfigurasi yang diinginkan aman atau benar secara semantik. Selain itu, versi apparatus, inventaris formal, batas waktu pengamatan, serta kriteria kesehatan belum diverifikasi dalam draf ini. Protokol juga belum memperoleh tinjauan pembimbing dan kemampuan apparatus untuk menjalankannya belum divalidasi. Seluruh keterbatasan tersebut perlu dipertimbangkan sebelum hasil empiris tersedia atau rekomendasi praktis dirumuskan.

DAFTAR PUSTAKA

R. Aghili, Q. Qin, H. Li, and F. Khomh. Understanding web application workloads and their applications: Systematic literature review and characterization. In *2024 IEEE International Conference on Web Services (ICWS)*. IEEE, 2024.

Argo Project. Argocd documentation, 2024. URL `https://argo-cd.readthedocs.io`.

Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. Borg, omega, and kubernetes. *ACM Queue*, 14(1):70–93, 2016.

Scott Chacon and Ben Straub. *Pro Git*. Apress, 2nd edition, 2014.

CNCF GitOps Working Group. Opengitops principles v1.0.0. `https://opengitops.dev`, 2021.

Thomas D. Cook and Donald T. Campbell. *Quasi-Experimentation: Design and Analysis Issues for Field Settings*. Houghton Mifflin, 1979.

M. D'Amore. Gitops and argocd: Continuous deployment and maintenance of a full stack application in a hybrid cloud kubernetes environment. Master's thesis, Politecnico di Torino, 2021.

Nicole Forsgren, Jez Humble, and Gene Kim. *Accelerate: The Science of Lean Software and DevOps*. IT Revolution Press, 2018.

Jez Humble and David Farley. *Continuous Delivery: Reliable Software Releases Through Build, Test, and Deployment Automation*. Addison-Wesley Professional, 2010.

Thomas A. Limoncelli, Strata R. Chalup, and Christina J. Hogan. *The Practice of Cloud System Administration: Designing and Operating Large Distributed Systems*. Addison-Wesley Professional, 2nd edition, 2018.

M. E. Matubber. Managing 5g kubernetes infrastructures using gitops principles. Master's thesis, KTH Royal Institute of Technology, 2025.

Kief Morris. *Infrastructure as Code: Dynamic Systems for the Cloud Age*. O'Reilly Media, 2nd edition, 2020.

P. Kagganti Nataraja. A security-centric analysis of declarative and imperative deployment approaches in kubernetes-based application environments. Master's thesis, National College of Ireland, 2025.

E. Paavola. Managing multiple applications on kubernetes using gitops principles. Master's thesis, Turku University of Applied Sciences, 2021.

O. Pohjola. Mitigating configuration drift in infrastructure-as-code systems. Master's thesis, Aalto University, 2025.

A. Rahman, S. I. Shamim, D. B. Bose, et al. Security misconfigurations in open source kubernetes manifests: An empirical study. *ACM Transactions on Software Engineering and Methodology*, 2023.

D. Schlette, P. Empl, M. Caselli, T. Schreck, and G. Pernul. Do you play it by the books? a study on incident response playbooks and influencing factors. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2024.

Michael L. Scott. *Programming Language Pragmatics*. Morgan Kaufmann, 4th edition, 2015.

R. Shrestha and A. A. N. Ali. Configuration management in kubernetes environments: A gitops approach. In *2024 IEEE/ACM 17th International Conference on Utility and Cloud Computing (UCC)*. IEEE, 2024.

H. Siddiqui, F. Khendek, and M. Toeroe. Microservices based architectures for iot systems—state-of-the-art review. *Internet of Things*, 2023.

Ervina Utami and Hanif Al Fatta. Analysis on the use of declarative and pull-based deployment models on gitops using argo cd. In *2021 4th International Conference on Information and Communications Technology (ICOIACT)*. IEEE, 2021.

Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. Large-scale cluster management at google with borg. In *Proceedings of the Tenth European Conference on Computer Systems (EuroSys '15)*, pages 1–17. ACM, 2015.

Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. *Experimentation in Software Engineering*. Springer, 2nd edition, 2012.

Billy Yuen, Alexander Matyushentsev, Jesse Suen, and Thomas Ekenstam. *GitOps and Kubernetes: Continuous Deployment with Argo CD, Jenkins X, and Flux*. O'Reilly Media, 2021. ISBN 978-1492094877.

Yue Zhang, Uchswas Paul, Marcelo d'Amorim, and Akond Rahman. Configuration defects in kubernetes. *IEEE Transactions on Software Engineering*, 52(3):1125–1144, 2026. doi: 10.1109/TSE.2026.3659785.

LAMPIRAN A

LAMPIRAN A

Lampiran ini disiapkan untuk artefak yang akan dipublikasikan setelah protokol ditinjau, apparatus divalidasi, dan pengumpulan data formal dilakukan. Pada draf ini, skrip, konfigurasi aktual, dan dataset tervalidasi belum dilampirkan. Ketiadaan artefak tersebut tidak menunjukkan bahwa pengujian telah berhasil atau gagal.

1.1 Artefak Eksperimen yang Direncanakan

Lampiran versi akhir dapat memuat sumber pemeriksa dan korektor imperatif berbasis `kubectl`, konfigurasi aplikasi ArgoCD, manifest dan inventaris workload formal, skema kolektor, serta prosedur pemulihan nilai dasar. Setiap artefak perlu diberi versi dan dikaitkan dengan konfigurasi aktual eksperimen. Materi pilot S1/S2 harus dipisahkan dari artefak dan data pengujian formal.

1.2 Skema Data dan Perlindungan Informasi

Skema log formal perlu menyimpan pengenal run, lingkungan, skenario pada berkas metadata terpisah dari masukan pengendali, status penerimaan injeksi, cap waktu monotonik dalam detik dan UTC untuk t_0, t_a, t_d, t_e, dan t_r, interval polling aktual, hasil kesetaraan dan kesehatan, status penyensoran, kesalahan, serta alasan run tidak valid. t_a menyimpan waktu penerimaan respons API injeksi. Metadata pengenal skenario tidak boleh diberikan kepada mekanisme pendeteksi atau korektor.

Snapshot dan bukti harus disanitasi sebelum disimpan atau dibagikan. Nilai Secret tidak dimasukkan ke listing, tangkapan layar, log, atau dataset publik. Pencocokan Secret dilakukan di memori; artefak yang dibagikan hanya memuat identitas bidang dan hasil kesetaraan, tanpa nilai rahasia atau hash yang dapat dipakai untuk menebak isinya.

1.3 Status Lampiran

Seluruh item pada lampiran ini masih berupa kebutuhan artefak, bukan skrip atau konfigurasi yang telah diverifikasi. Isi lampiran hanya dapat dilengkapi berdasarkan bukti yang benar-benar dihasilkan oleh eksperimen dan telah ditinjau untuk

mencegah kebocoran informasi sensitif.